

# Minimum Spanning Tree

→ Algorithme de Kruskal (MST)

↳ une écriture de l'algorithme de Kruskal.

Fonction  $\text{Kruskal-MST}(G = (S, A, w))$

*% Initialisation.*  
{ Trier les arêtes du graphe  $G$  par ordre croissant  
en une liste  $L$ .

$F \leftarrow \{\}$ .

*% Boucle Principale.*

Tant que  $|F| < |S| - 1$  et  $L \neq \emptyset$  faire {  
 $e \leftarrow \text{Premier}(L)$

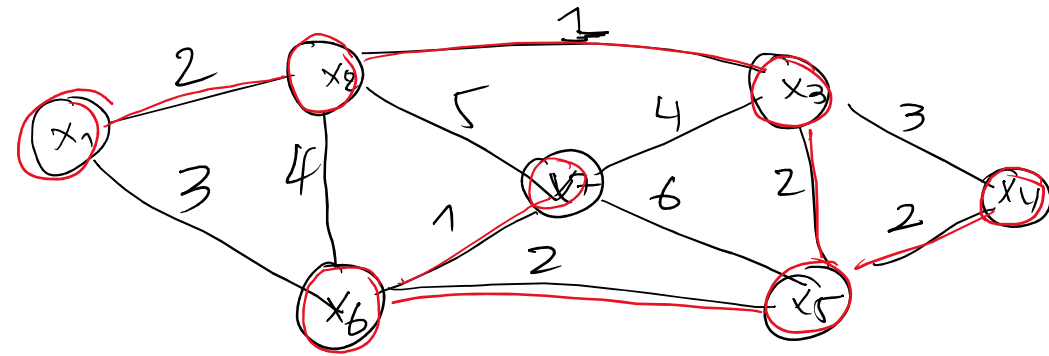
Extraire  $(L, e)$

*" L'ajout de  $e$  à  $F$  "* Si  $T = (S, F \cup \{e\})$  n'a pas de cycle alors  
*→*  $F \leftarrow F \cup \{e\}$ .  
Retourner  $F$ .

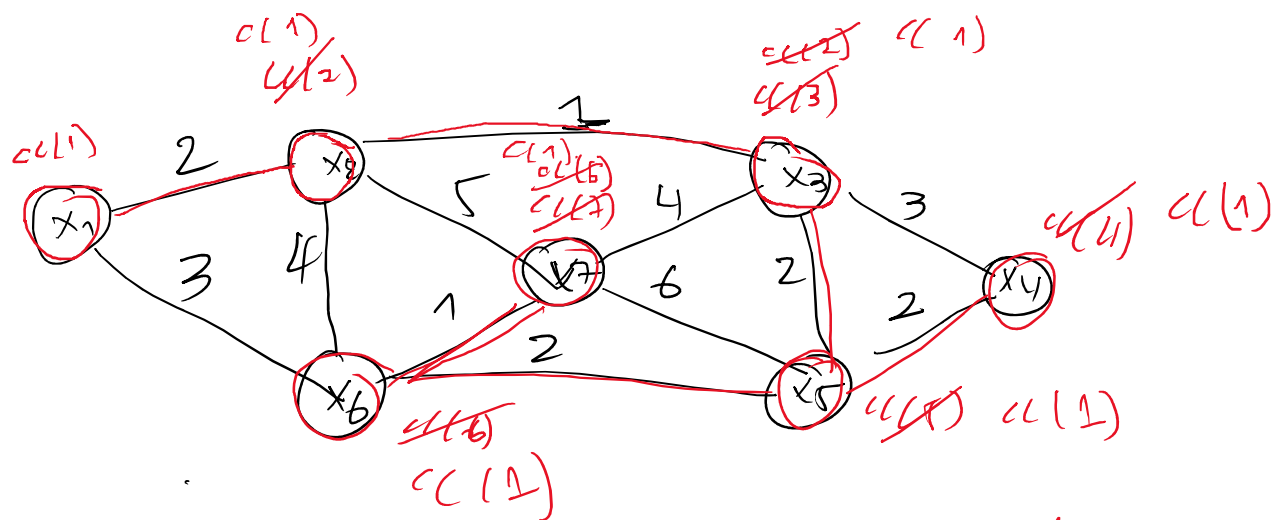
→ simulation de l'algorithme Kruskal.

Considérons le graphe valué  $G = (S, A, w)$  suivant;

| arêtes triées<br>par ordre croissant | arêtes retenues |
|--------------------------------------|-----------------|
| <del><math>x_2 x_3</math></del>      | ✓               |
| <del><math>x_6 x_7</math></del>      | ✓               |
| <del><math>x_1 x_2</math></del>      | ✓               |
| <del><math>x_3 x_5</math></del>      | ✓               |
| <del><math>x_1 x_5</math></del>      | ✓               |
| <del><math>x_5 x_6</math></del>      | ✓               |
| <del><math>x_1 x_6</math></del>      |                 |
| <del><math>x_3 x_4</math></del>      |                 |
| <del><math>x_2 x_6</math></del>      |                 |
| <del><math>x_3 x_7</math></del>      |                 |
| <del><math>x_2 x_7</math></del>      |                 |
| <del><math>x_5 x_7</math></del>      |                 |



- Question: proposer une  
procédure permettant  
de détecter les cycles.



Modifiez l'algorithme de Kruskal en ajoutant cette procédure permettant de détecter les cycles.

→ Algorithme de plus courts chemins (PCC)

↳ Différents problèmes du PCC.

Pour un graphe  $G=(S, A, w)$  orienté valué

$\mathcal{P}_1$ : Étant donné deux sommets  $x$  et  $y$ , trouver le plus court chemin de  $x$  à  $y$ .

$\mathcal{P}_2$ : Étant donné un sommet de départ  $x_0$ , pour tout  $x \in S$ , trouver le plus court chemin allant de  $x_0$  à  $x$ .

$\mathcal{P}_3$ : Pour tout  $x, y \in S'$ , trouver le plus court chemin de  $x$  à  $y$ .

Dans l'énumération précédente, la complexité du problème est croissante

$$C(P_1) \leq C(P_2) \leq C(P_3)$$

un algorithme qui peut résoudre  $P_3$  peut aussi résoudre  $P_2$  et  $P_1$  et un algorithme qui résout  $P_2$  peut aussi résoudre  $P_1$ .

↳ le cas des valuations (poids) positives.

→ Algorithme de Dijkstra (1959)

- Données : - un graphe orienté pondéré, une fonction de coût  $c: A \rightarrow \mathbb{R}^+$
- Résultat : Arborecence des Plus courts chemins issus du sommet de départ  $x_0$ .

↳ une écriture de l'algorithme de Dijkstra.

Dijkstra ( $G = (S, A, w)$ ,  $x_0 \in S$ )

$\left\{ \begin{array}{l} \text{OUVERT} \leftarrow \{x_0\} \\ \text{FERME} \leftarrow \emptyset \end{array} \right.$

Pour chaque sommet  $x \in \mathcal{V}$  faire {

$\pi[x] \leftarrow \text{NIL}$

$d[x] \leftarrow \infty$  }

" cette écriture contient  
2 grands problèmes !!

$d[x_0] \leftarrow 0$  ;

% Boucle principale

Tant que OUVERT  $\neq \emptyset$  faire {

$sc \leftarrow \text{Extraire\_minimum}(\text{OUVERT})$ .

$\text{FERME} \leftarrow \text{FERME} \cup \{sc\}$

Pour chaque sommet  $y \in \text{succ}(sc)$  faire {

Si  $d[sc] + c(sc, y) < d[y]$  alors {

$d[y] \leftarrow d[sc] + c(sc, y)$

$\pi[y] \leftarrow sc$  } }

Retourner  $\pi$

Procédure  
de relaxation.

↳ le cas des valuations (poids) positives.

→ Algorithme de Dijkstra (1959)

- Données : - un graphe orienté pondéré, une fonction de coût  $c: A \rightarrow \mathbb{R}^+$
- Résultat : Arborecence des Plus courts chemins issus du sommet de départ  $x_0$ .

L'algorithme de Dijkstra utilise deux procédures de base à savoir :

Procédure Initialisation - Pre  $\{ G(S, A, w), x_0 \in S \}$ .

} Pour chaque sommet  $x \in S$  faire  $\{$   
 $\pi[x] \leftarrow NIL, d[x] \leftarrow \infty \}$ .  
}

Procédure Relaxation-Proc ( $(x, y) \in A$ )

{ si  $d[y] > d[x] + c(x, y)$  alors

{  $d[y] \leftarrow d[x] + c(x, y)$

$\pi[y] \leftarrow x$  }

Une écriture de l'algorithme de Dijkstra

Dijkstra( $G = (S, A, w)$ ,  $x_0 \in S$ ) {

% Initialisation.

Initialisation-Proc( $G(S, A, w)$ )

FERME  $\leftarrow \emptyset$

OUVERT  $\leftarrow \{x_0\}$ .

$d[x_0] \leftarrow 0$ .

% Bonne Principale.

Tant que  $OUVERT \neq \emptyset$  faire {  
   $x \leftarrow \text{ExtraireMinimum}(OUVERT)$   
   $FERME \leftarrow FERME \cup \{x\}$ .

Pour chaque  $y \in \text{succ}(x)$  faire {

  si  $y \notin FERME$  alors {

$\text{relaxation}(\pi, y)$

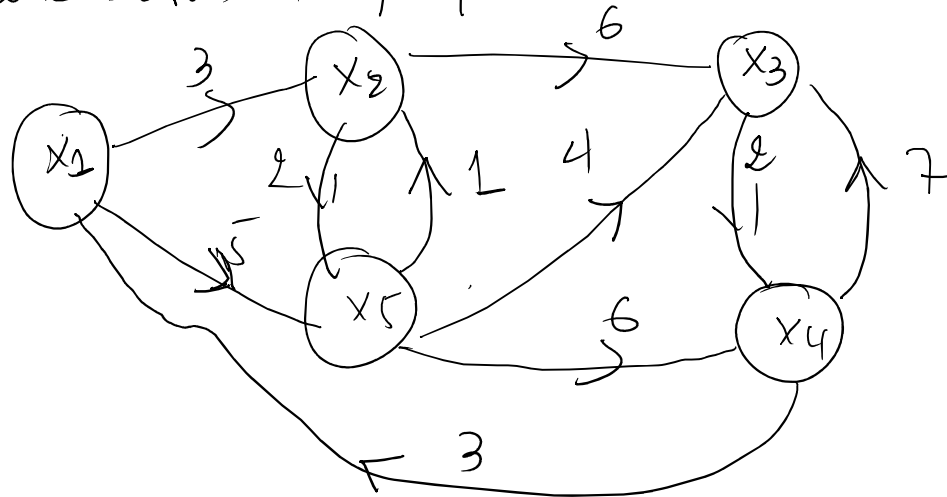
  si  $y \notin OUVERT$  alors

$OUVERT \leftarrow OUVERT \cup \{y\}$ .

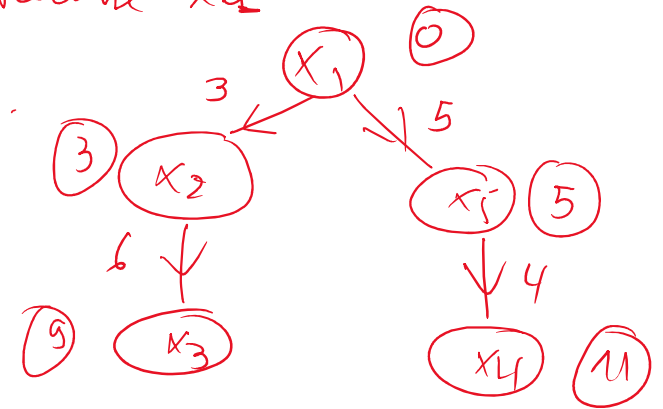
}  
}  
}

Retourner  $\pi, d$ .

→ simulation de l'algorithme :  
 Considérons le graphe suivant :



D'où l'absence des PC(s)  
 de racine  $x_1$



| Sommet | $x_1$                              | $x_2$                              | $x_3$                              | $x_4$                               | $x_5$                              |
|--------|------------------------------------|------------------------------------|------------------------------------|-------------------------------------|------------------------------------|
| SD     | -                                  | <del><math>x_1</math></del>        | <del><math>x_2</math></del>        | <del><math>x_5</math></del>         | <del><math>x_1</math></del>        |
| $\pi$  | -                                  |                                    |                                    |                                     |                                    |
| $d$    | <del><math>+ \infty</math></del> 0 | <del><math>+ \infty</math></del> 3 | <del><math>+ \infty</math></del> 9 | <del><math>+ \infty</math></del> 11 | <del><math>+ \infty</math></del> 5 |
| OUVERT | <del><math>x_1</math></del>        | <del><math>x_2</math></del>        | <del><math>x_5</math></del>        | <del><math>x_3</math></del>         | <del><math>x_4</math></del>        |
| FERME  | $(x_2, -, 0)$                      | $(x_2, x_1, 3)$                    | $(x_5, x_1, 5)$                    | $(x_3, x_2, 9)$                     | $(x_4, x_5, 11)$                   |

**Problème 2 :**

Une société gère un réseau autoroutier reliant 8 villes que l'on notera *A, B, C, D, E, F, G* et *H*. Ce réseau est donné par la matrice suivante :

|   | A        | B        | C        | D        | E        | F        | G        | H        |
|---|----------|----------|----------|----------|----------|----------|----------|----------|
| A | 0        | 8        | 6        | 5        | $\infty$ | $\infty$ | 6        | 15       |
| B | 8        | 0        | $\infty$ | 10       | 14       | 8        | 10       | $\infty$ |
| C | 6        | $\infty$ | 0        | 5        | $\infty$ | $\infty$ | $\infty$ | 8        |
| D | 5        | 10       | 5        | 0        | 6        | $\infty$ | $\infty$ | $\infty$ |
| E | $\infty$ | 14       | $\infty$ | 6        | 0        | 10       | $\infty$ | $\infty$ |
| F | $\infty$ | 8        | $\infty$ | $\infty$ | 10       | 0        | 12       | $\infty$ |
| G | 6        | 10       | $\infty$ | $\infty$ | $\infty$ | 12       | 0        | 12       |
| H | 15       | $\infty$ | 8        | $\infty$ | $\infty$ | $\infty$ | 12       | 0        |

Dans cette matrice  $M(i, j)$  représente le coût du péage (l'unité 10 DH) de l'autoroute reliant la ville  $i$  à la ville  $j$ . On a évidemment  $M(i, i) = 0$ , et, lorsque  $M(i, j) = \infty$ , cela signifie qu'il n'y a pas d'autoroute reliant directement  $i$  à  $j$  quand  $i \neq j$ .

- 1) Une agence de tourisme située en *A*, veut déterminer les itinéraires les plus économiques vers chacune des autres villes. A quel type de problème formel peut-on associer cette situation ? Préciser les algorithmes vus en cours qui peuvent résoudre ce problème.
- 2) Appliquer l'algorithme adéquat pour résoudre le problème modélisant la situation décrite par la question 1).
- 3) A quoi correspond la solution retournée par l'algorithme que vous avez appliqué ? dessiner graphiquement cette solution. Quel est alors l'itinéraire le plus économique reliant *A* à *H*.